

MLOps Pipeline: Breast Cancer Classification

Architecture MLOps Complète

Docker • MLflow • MinIO • PostgreSQL • DVC • Discord Notifications

 LABIADH LINA

Cours DevOps - MLOps • Mai 2026

Docker

MLflow

MinIO

DVC



Sommaire du Rapport

01

Contexte & Objectifs

MLOps, cycle de vie ML, enjeux du projet

02

Architecture & Infrastructure

Docker Compose, 4 services, réseau interne

03

Versioning des Données (DVC)

dvc init, remote MinIO, push/pull dataset

04

Expérimentation MLflow

Autolog, tracking, artifacts dans MinIO

05

Notifications Discord

Webhook, embeds, niveaux info/success/error

06

Résultats & Démonstration

Screenshots, métriques, runs MLflow

07

Conclusion & Perspectives

CI/CD GitLab, monitoring, améliorations

01 • Contexte & Objectifs

Qu'est-ce que le MLOps ?

Le MLOps (Machine Learning Operations) combine DevOps et Machine Learning pour automatiser, fiabiliser et industrialiser le cycle de vie complet des modèles IA — de l'expérimentation locale jusqu'au déploiement en production.

Objectif Principal

Mettre en place une infrastructure MLOps complète et fonctionnelle avec Docker, MLflow, MinIO et PostgreSQL.

Reproductibilité

Versionner les données avec DVC et les modèles avec MLflow pour garantir la reproductibilité de chaque expérience.

Monitoring Temps Réel

Intégrer des notifications Discord automatiques pour suivre l'entraînement, les métriques et les erreurs.

02 • Architecture & Infrastructure Docker

 **mlops_net** (réseau Docker interne)



PostgreSQL 14

:5432

Backend store
MLflow metadata



MinIO

:9000/9001

Artifact store
S3-compatible



MLflow Server

:5000

Tracking server
Experiments UI



Jupyter Lab

:8888

Data science
environment

depends_on →

depends_on →

02 • Dockerfiles du Projet

Dockerfile.jupyter

```
FROM jupyter/datascience-notebook:latest

COPY requirements.txt /tmp/

RUN pip install -r /tmp/requirements.txt
```

requirements.txt inclus :

- mlflow
- jupyterlab
- scikit-learn
- pandas / numpy
- boto3
- psycopg2-binary
- requests

Dockerfile.mlflow

```
FROM python:3.9-slim

RUN apt-get update && \

    apt-get install -y libpq-dev gcc

RUN pip install mlflow boto3 \

    psycopg2-binary

CMD mlflow server \

    --backend-store-uri postgresql://\

    root:root@postgres:5432/mlflow \

    --default-artifact-root s3://mlflow \

    --host 0.0.0.0
```


You, yesterday | 1 author (You) | ↓ Pull Images | ▷ Compose Up
version: '3.8'

▷ Run All Services

services:

▷ Run Service

```
postgres:
  image: postgres:14
  container_name: mlops_postgreslab
  environment:
    POSTGRES_USER: root
    POSTGRES_PASSWORD: root
    POSTGRES_DB: mlflow
  ports:
    - "5432:5432"
  volumes:
    - pg_data:/var/lib/postgresql/data
  networks:
    - mlops_net
```

▷ Run Service

```
minio:
  image: minio/minio:latest
  container_name: mlops_miniolab
  environment:
    MINIO_ROOT_USER: root
    MINIO_ROOT_PASSWORD: root123456789
  ports:
    - "9010:9000" # On change - "9000:9000"
    - "9001:9001"
  command: server /data --console-address ":9001"
  volumes:
    - minio_data:/data
  networks:
    - mlops_net
```

▷ Run Service

```
mlflow:
  build:
    context: .
    dockerfile: docker/Dockerfile.mlflow
  container_name: mlops_mlflowlab
  ports:
    - "5000:5000"
  environment:
    AWS_ACCESS_KEY_ID: root
    AWS_SECRET_ACCESS_KEY: root123456789
    MLFLOW_S3_ENDPOINT_URL: http://minio:9000
  depends_on:
    - postgres
    - minio
  networks:
    - mlops_net
```

▷ Run Service

```
jupyter:
  build:
    context: .
    dockerfile: docker/Dockerfile.jupyter
  container_name: mlops_jupyterlab
  ports:
    - "8888:8888"
  environment:
    JUPYTER_ENABLE_LAB: "yes"
    JUPYTER_TOKEN: "mlflow"
    AWS_ACCESS_KEY_ID: root
    AWS_SECRET_ACCESS_KEY: root123456789
    MLFLOW_S3_ENDPOINT_URL: http://minio:9000
  volumes:
    - ./:/home/jovyan/work
  networks:
    - mlops_net
```



Docke-compose.yml

• Lancement des conteneurs:

Commande exécutée dans le terminal PowerShell depuis le dossier du projet :

docker compose up -d -build

→ Cette commande construit les images Docker et démarre les 4 conteneurs en arrière-plan.

```
(.venv) PS C:\Users\linal_4qqkavn\Desktop\lab0_sentiment-api> docker-compose up -d
time="2026-05-11T20:23:04+01:00" level=warning msg="C:\\Users\\linal_4qqkavn\\Desktop\\lab0_se
e ignored, please remove it to avoid potential confusion"
[+] up 5/5
✔ Network lab0_sentiment-api_mlops_net Created 0.1s
✔ Container mlops_jupyterlab Created 0.4s
✔ Container mlops_postgreslab Created 0.4s
✔ Container mlops_miniolab Created 0.3s
✔ Container mlops_mlflowlab Created 0.2s
```

```
(.venv) PS C:\Users\linal_4qqkavn\Desktop\lab0_sentiment-api> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
9a8fa02bbc46   lab0_sentiment-api-mlflow           "/bin/sh -c 'mlflow ..." About an hour ago Up About an hour 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
b4f621fcd426   lab0_sentiment-api-jupyter          "tini -g -- start-no..." About an hour ago Up About an hour (healthy) 0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
73eb8882a1e7   postgres:14                         "docker-entrypoint.s..." About an hour ago Up About an hour 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
c2e0acd880d5   minio/minio:latest                 "/usr/bin/docker-ent..." About an hour ago Up About an hour 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp, 0.0.0.0:90
10->9000/tcp, [::]:9010->9000/tcp
(.venv) PS C:\Users\linal_4qqkavn\Desktop\lab0_sentiment-api>
```

03 • Versioning des Données avec DVC

Pourquoi DVC ?

Git ne gère pas les gros fichiers de données.
DVC versionne les datasets et les stocke
dans un remote S3/MinIO.

`.dvc/config` — Remote MinIO

```
[core] remote = myremote
```

```
['remote "myremote"']
```

```
url = s3://mlflow/
```

```
endpointurl = http://127.0.0.1:9010
```

```
access_key_id = root
```

```
secret_access_key = root123456789
```

1

```
py -3.14 -m dvc init
```

Initialise DVC dans le repo Git

2

```
dvc remote add -d myremote s3://mlflow/dvcstore
```

Définit MinIO comme remote par défaut

3

```
dvc remote modify myremote endpointurl ...
```

Configure l'URL du endpoint MinIO local

4

```
dvc add data/dataset.csv
```

Crée dataset.csv.dvc et ignore le CSV dans Git

5

```
git add data/dataset.csv.dvc .gitignore
```

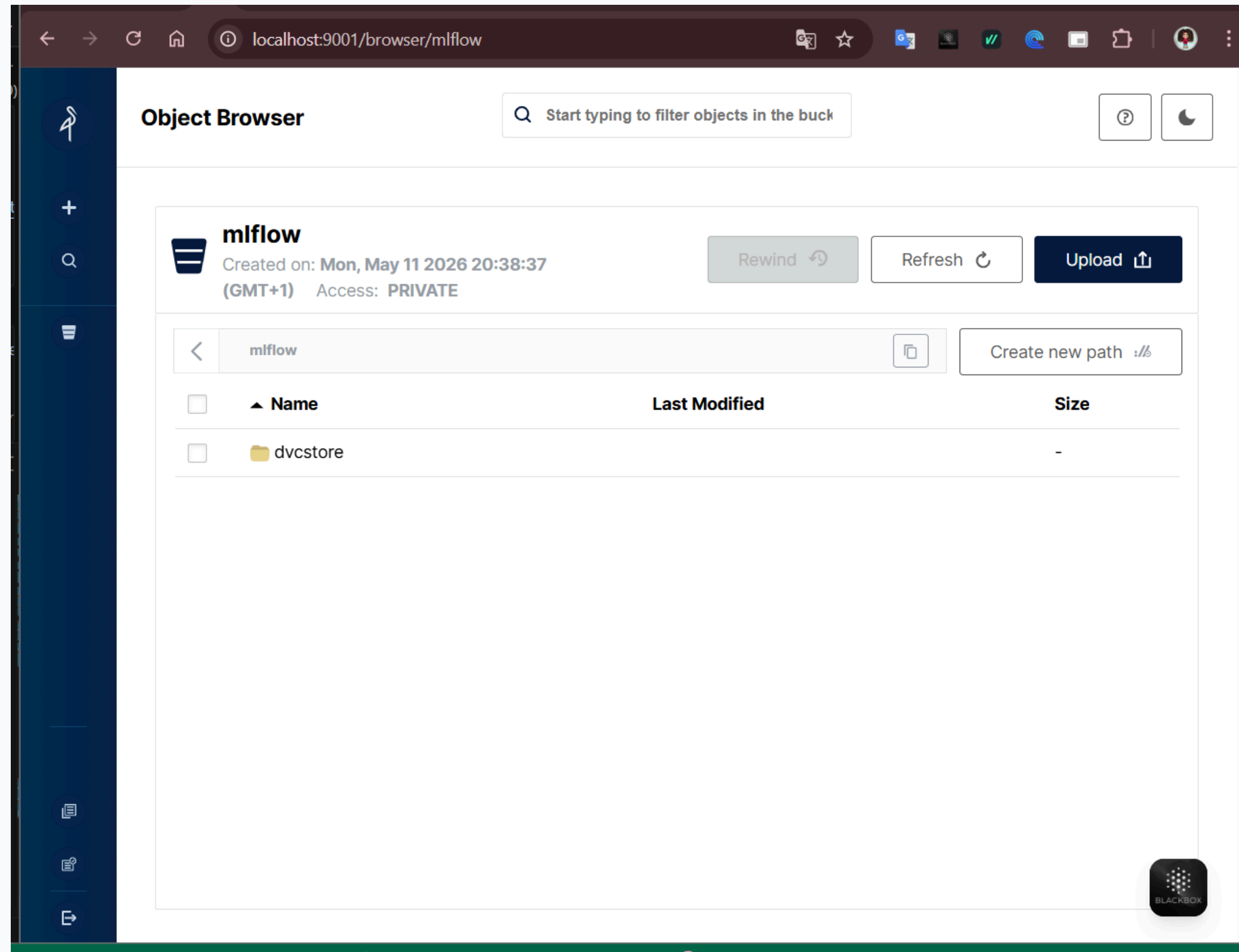
Versionne le pointeur DVC dans Git

6

```
dvc push
```

Envoie le dataset vers MinIO (dossier dvcstore)

03 • Versioning des Données avec DVC



interface MinIO → Bucket mlflow(dossier dvcstore)

bucket MinIO

- Ouvrir `http://localhost:9001` dans le navigateur
- Se connecter avec les identifiants : `root/root123456789`

04 • Expérimentation MLflow

01_experimentation.ipynb

```
# 1. Connexion MLflow

mlflow.set_tracking_uri('http://mlflow:5000')
mlflow.set_experiment('Classification_Cancer')
mlflow.sklearn.autolog() # ← log automatique !

# 2. Entraînement dans un Run context
with mlflow.start_run() as run:
    clf = RandomForestClassifier(
        n_estimators=100, max_depth=5)
    clf.fit(X_train, y_train)
    preds = clf.predict(X_test)
    acc = accuracy_score(y_test, preds)

# 3. Notification Discord
notify_discord(message, metrics={'Accuracy': acc},
               level='success', task_name='Training')
```

Ce que MLflow capture automatiquement :



Hyperparamètres

n_estimators, max_depth, max_features...



Métriques

accuracy, precision, recall, F1, AUC...



Artefacts/Modèle

.pkl sauvegardé dans MinIO via S3



Dataset

Référence train/test loggée



Run ID unique

Hash pour recharger le modèle exact

05 • Notifications Discord Automatiques

 discord_notify.py — src/utils/

```
WEBHOOK_URL = os.getenv('DISCORD_WEBHOOK_URL')

def notify_discord(message, run_id='N/A',
                  metrics=None, level='success',
                  task_name='Training'):

    colors = {
        'info':3447003,    # Bleu
        'success':3066993, # Vert
        'error':15158332   # Rouge
    }

    payload = { 'username':'Bot-Notifier-MLOps',
                'embeds':[{ 'title':..., 'fields':[
                    Environnement, Auteur, Run ID,
                    Métriques, MLflow UI link
                ]}]
    }

    requests.post(WEBHOOK_URL, json=payload)
```

3 niveaux de notification :



INFO

Début de session Jupyter,
début d'entraînement



SUCCESS

Entraînement terminé,
modèle sauvegardé dans MinIO



ERROR

Exception lors de
l'entraînement (traceback)

 Webhook stocké dans variable d'env. DISCORD_WEBHOOK_URL

06 • Résultats & Démonstration

Runs MLflow

4

Classification_Cancer

Meilleure Accuracy

92.98%

RandomForest n=100 d=5

Artefacts MinIO

4.7 MiB

29 objets dans le bucket

Notifs Discord

3/3

info + success + error

 Runs enregistrés dans MLflow (Classification_Cancer)

Run Name	Créé	Durée	Modèle	Statut
vaunted-pig-402	2 min ago	8.1s	RandomForest	✓ Completed
bold-wolf-897	17 min ago	9.5s	RandomForest	✓ Completed
bright-asp-601	23 min ago	9.0s	RandomForest	✓ Completed
intelligent-crab-735	51 min ago	13.4s	RandomForest	✓ Completed

06 • Résultats & Démonstration

🧪 Résultats dans l'interface MLflow (Classification_Cancer) → Liste des runs { localhost:5000 }

mlflow 3.1.4 Experiments Models Prompts

localhost:5000/#/experiments/1?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&m... ☆

Experiments (+) (-)

Search experiments

- ☐ Default
- ☒ Classification_Cancer

Classification_Cancer ⓘ Provide Feedback Add Description Share

Runs Models Experimental Evaluation ⓘ Traces

metrics.rmse < 1 and params.model = "tree" ⓘ Time created ▾ State: Active ▾ Datasets ▾ + New run

Sort: Created ▾ Columns ▾ Expand rows Group by ▾

	Run Name	Created ↕	Dataset	Duration	Source	Models
☐	vaunted-pig-402	✓ 38 seconds ago	dataset (8c894dfc) Train , dataset (7d1...	8.1s	01_exper...	model
☐	bold-wolf-897	✓ 15 minutes ago	dataset (2317d30c) Train , dataset (66e...	9.5s	01_exper...	model
☐	bright-asg-601	✓ 20 minutes ago	dataset (6198dcc4) Train , dataset (aca...	9.0s	01_exper...	model
☐	intelligent-crab-735	✓ 49 minutes ago	dataset (32390cf9) Train , dataset (e59...	13.4s	01_exper...	model

06 • Résultats & Démonstration

Résumé des métriques obtenues

✓

MLOPS LAB - RAPPORT AUTOMATIQUE

Le modèle a été entraîné et sauvegardé avec succès dans MinIO.

Auteur

ING. LABIADH LINA

Statut

Succès

Run ID

9d6e64dbea3f461ca40bb8fe96755ef1

n_estimators

100

max_depth

6

max_features

3

Métriques

Accuracy: 0.9386

Precision: 0.9286

Recall: 0.9701

F1-Score: 0.9489

MLflow UI

Accéder à l'interface

MLOps Lab • RandomForestClassifier • Cancer Dataset • ING. LINA • Aujourd'hui à 23:05

PIPELINE-START

Pipeline MLOps démarré avec succès sur Docker.

Auteur

ING. LABIADH LINA

Statut

Échec

Run ID

N/A

Métriques

N/A

MLflow UI

Accéder à l'interface

MLOps Lab • RandomForestClassifier • Cancer Dataset • ING. LINA • Aujourd'hui à 23:05

TRAINING

Entraînement en cours (Run: 9d6e64dbea3f461ca40bb8fe96755ef1)...

Auteur

ING. LABIADH LINA

Statut

Échec

Run ID

N/A

Métriques

N/A

MLflow UI

Accéder à l'interface

MLOps Lab • RandomForestClassifier • Cancer Dataset • ING. LINA • Aujourd'hui à 23:05

Vérification du stockage dans MinIO

Object Browser

Start typing to filter objects in the bucket

mlflow

Created on: Mon, May 11 2026 20:38:37 (GMT+1) Access: PRIVATE 4.7 MiB - 29 Objects

mlflow

Name

Last Modified

1

dvcstore

mlflow

Created on: Mon, May 11 2026 20:38:37 (GMT+1) Access: PRIVATE 7.2 MiB - 99 Objects

mlflow / 1

Name

Last Modified

models

dd2905274ef9459faf893ff88ebbe5af

c29b66f9e8da43e4b30d3ac52bea3a32

b7d2c2aa396c49a5a03853720c98eb2d

9d6e64dbea3f461ca40bb8fe96755ef1

07 • Conclusion & Perspectives

✓ Réalisations du Lab

- Infrastructure Docker complète (4 services)
- DVC configuré → dataset versionné dans MinIO
- MLflow tracking avec autolog activé
- 4 runs enregistrés avec accuracy ~92%
- Artefacts (.pkl) stockés dans MinIO
- Bot Discord avec 3 niveaux de notification
- Notebook Jupyter fully dockerisé

🚀 Perspectives & Améliorations

- CI/CD GitLab — pipeline auto train + deploy
- Monitoring Prometheus + Grafana (Data Drift)
- Déploiement API FastAPI / Flask
- Model Registry MLflow (alias @production)
- Tests unitaires pytest automatisés
- Lab 2 : NLP avec DistilBERT & Transformers

Stack Technologique Maîtrisée :

Docker Compose

PostgreSQL

MinIO S3

MLflow 3.1

DVC

scikit-learn

Discord Webhooks

Python 3.11